COVID-19 Chest X-ray Classification on Azure ML: Project Report

Aditi Srivastava · 25 September 2026

Abstract

I built an image classification model that sorts chest X-rays into **Covid**, **Normal** or **Viral Pneumonia**, trained and deployed entirely on **Microsoft Azure Machine Learning**. The model uses **transfer learning** on a ResNet-50 network pretrained on ImageNet, fine-tuned on 251 labelled X-rays. On 66 unseen test images it reached **93.94% accuracy**.

The trained model was registered in Azure ML and deployed as a **managed online endpoint**, a REST API that accepts any new X-ray and returns the predicted class with a confidence score. The live endpoint classified 6 out of 6 sample test images correctly, and a newly supplied X-ray was classified as Viral Pneumonia with 96.5% confidence.

This is an educational project, not a medical diagnostic tool.

1. Problem statement and objectives

Problem statement: Given a chest X-ray image, automatically classify it as Covid, Normal or Viral Pneumonia, and make this classifier available as a cloud service that any application can call.

Why this problem matters

- Covid-19 and viral pneumonia both affect the lungs and can look similar on an X-ray, so telling them apart takes expert radiologists.
- During outbreaks, hospitals get far more X-rays than radiologists can read quickly.
- A model that gives a fast first opinion can help prioritise which scans a doctor should look at first.
- Building it on the cloud means it can be used from anywhere, without the user needing a powerful computer.

Objectives

1. Prepare and explore a labelled chest X-ray dataset with three classes.
2. Train a deep learning image classifier on Azure Machine Learning using transfer learning.
3. Evaluate it on unseen test images using accuracy, precision, recall, F1-score and a confusion matrix.
4. Register the trained model in the Azure ML model registry, so it is versioned and reusable.
5. Deploy it as a real-time REST endpoint.
6. Show that the endpoint can classify a brand-new X-ray image it has never seen.

Type of problem: supervised learning, multi-class image classification (3 classes).

2. Dataset

The dataset (`Covid19-dataset.zip`, about 165 MB) holds 317 chest X-ray images, already split into a training folder and a test folder, with one sub-folder per class.

Class	Training images	Test images
Covid	111	26
Normal	70	20
Viral Pneumonia	70	20
Total	**251**	**66**

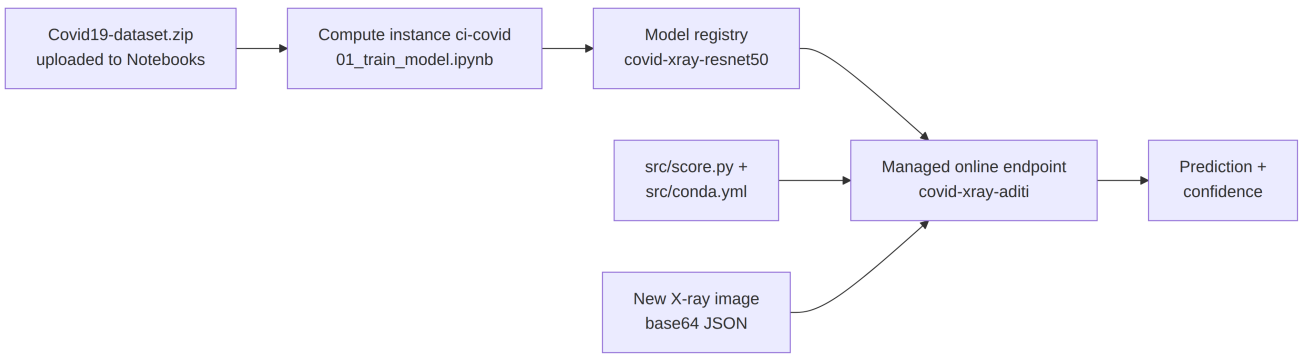
```
Covid19-dataset/
├── train/
│   ├── Covid/
│   ├── Normal/
│   └── Viral Pneumonia/
└── test/
    ├── Covid/
    ├── Normal/
    └── Viral Pneumonia/
```

Observations that shaped the design

- **Small dataset:** only 251 training images, too few to train a deep network from scratch. This is why I used transfer learning and data augmentation.
- **Class imbalance:** Covid has about 1.6 times as many images as each other class. Without correction the model would lean towards predicting Covid, so I used class weights in the loss.
- **Mixed formats:** images are a mix of JPEG and PNG. 267 are RGB, 42 are grayscale and 8 are RGBA, so every image is converted to 3-channel RGB before use.
- **Validation set:** there is no separate validation folder, so I held out 15% of the training images (38 images) for validation, keeping 213 for training. The test set is used only once, for the final evaluation.

3. Tools and Azure architecture

Everything ran on Azure Machine Learning under an Azure for Students subscription. Training happened on a cloud compute instance, the model was stored in the model registry, and predictions are served by a managed online endpoint.



The data flows left to right: train on the compute instance, register the model, then deploy it with the scoring script so new images can be sent to it.

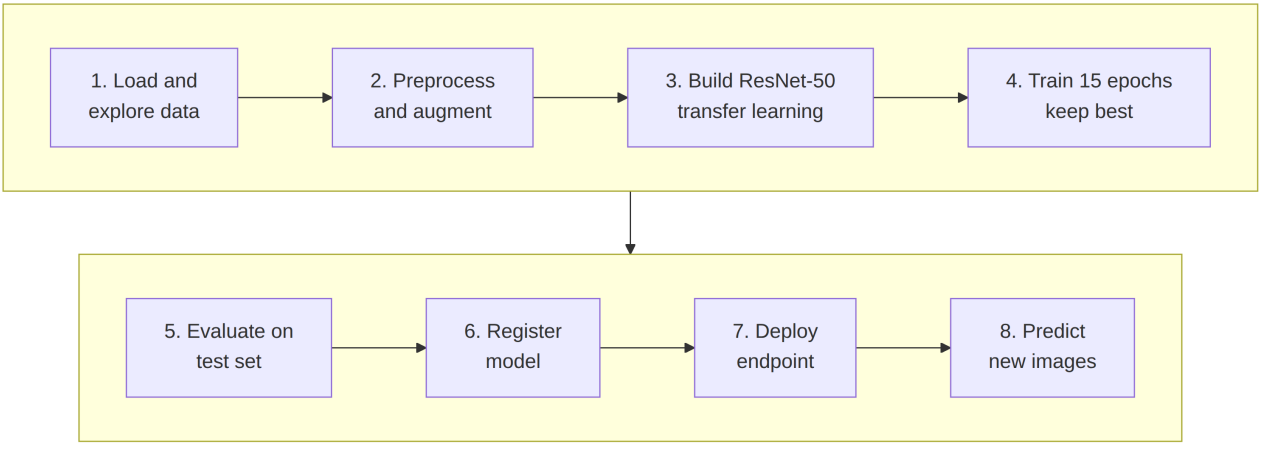
Component	What I used	Why
Workspace	`image_classification_ml` in resource group `Covid_19`	The central place that holds notebooks, compute, models and endpoints
Compute instance	`ci-covid`, Standard_DS3_v2 (4 vCPU, 14 GB RAM, CPU only)	A cloud machine to run Jupyter notebooks and train the model; a GPU was not needed for 251 images
Language and libraries	Python 3.10, PyTorch, torchvision, scikit-learn, matplotlib, seaborn	PyTorch for the neural network; scikit-learn for the train/validation split and metrics; matplotlib/seaborn for charts
Azure SDK	`azure-ai-ml` (SDK v2), `azure-identity`	To register the model and create the endpoint from Python code
Model registry	Azure ML Models, `covid-xray-resnet50` (versioned)	Keeps each trained model with a version number and its test accuracy as a tag
Endpoint	Managed online endpoint `covid-xray-aditi`, deployment `blue`, Standard_DS1_v2	Turns the model into a REST API that any app can call over HTTPS with a key
Environment	Ubuntu base image + `src/conda.yml` (CPU PyTorch, inference server)	The software the endpoint container needs to run the model

Project files

- `01_train_model.ipynb`: loads data, trains, evaluates, saves and registers the model.
- `02_deploy_endpoint.ipynb`: deploys the model, tests it and predicts new images.
- `src/score.py`: the code that runs inside the endpoint for every request.
- `src/conda.yml`: the Python packages for the endpoint.
- `outputs/`: saved charts, the classification report and the model files.

4. Methodology

The approach is transfer learning: start from a network that already understands images, then teach only its last layers to recognise lung conditions.



4.1 Preprocessing

- **Convert to RGB:** the pretrained network expects 3 colour channels, and some X-rays are grayscale or RGBA.

- **Resize to 224 × 224 pixels:** the input size ResNet-50 was trained on.
- **Normalise** with the ImageNet mean `[0.485, 0.456, 0.406]` and standard deviation `[0.229, 0.224, 0.225]`, so the pixel values match what the pretrained weights expect.

4.2 Data augmentation (training images only)

Augmentation creates slightly different versions of each training image every epoch. This lets a small dataset behave like a bigger one and reduces overfitting.

- **Random resized crop** (80–100% of the image): the model learns not to depend on exact framing.
- **Random rotation ±10°:** patients are not always positioned perfectly straight.
- **Brightness and contrast jitter ±20%:** X-ray machines produce different exposure levels.
- **No horizontal flip:** the heart sits on the left side of the chest, so flipping would create unrealistic X-rays.

Validation and test images are only resized and normalised, so they are evaluated as they really are.

4.3 Model: ResNet-50 with transfer learning

- **ResNet-50** is a 50-layer convolutional neural network (CNN). Its skip connections let very deep networks train well.
- It was **pretrained on ImageNet** (about 1.2 million images, 1,000 classes), so its early layers already detect edges, textures and shapes.
- **Frozen layers:** all layers except the last residual block (`layer4`) are frozen. This keeps the general visual knowledge and avoids overfitting on 213 training images.
- **New classifier head:** the original 1,000-class output layer is replaced by `Dropout(0.3) → Linear(2048 → 3)`. Dropout randomly switches off 30% of features during training, another guard against overfitting.
- **Trainable parameters:** 14,970,883 of 23,514,179 in total (about 64%).

Why transfer learning instead of training from scratch? Training a CNN from scratch needs tens of thousands of images. Reusing pretrained features gives high accuracy with only a few hundred.

4.4 Training setup

Setting	Value	Reason
Loss function	Cross-entropy with class weights (Covid 0.755, Normal 1.203, Viral Pneumonia 1.183)	Mistakes on the smaller classes cost more, which balances the Covid-heavy data
Optimiser	AdamW, learning rate 0.0001, weight decay 0.0001	Adam adapts the step size per parameter; a small learning rate avoids destroying pretrained weights
Learning-rate schedule	Cosine annealing over 15 epochs	The step size shrinks smoothly, so training settles into a good minimum
Epochs	15	Enough for the small dataset to converge without overfitting
Batch size	16	Fits comfortably in CPU memory
Model selection	Keep the weights with the best validation accuracy	Uses the validation set to pick the best epoch, not the last one
Random seed	42	Makes the results repeatable (retraining gave the same 93.94%)

4.5 Evaluation metrics

- **Accuracy:** share of test images classified correctly.
- **Precision:** of the images predicted as a class, how many really were that class.
- **Recall:** of the images that really were a class, how many the model found. This matters most in medicine, because a missed Covid case is worse than a false alarm.
- **F1-score:** the balance between precision and recall.
- **Confusion matrix:** shows exactly which classes get mixed up.

5. Code walkthrough: training notebook (01_train_model.ipynb)

The notebook runs top to bottom on the compute instance. The important cells are below, each with what it does and why.

Step 1: configuration

```
DEVICE = torch.device("cuda" if torch.cuda.is_available() else "cpu")

IMG_SIZE   = 224     # ResNet-50 input size
BATCH_SIZE = 16
EPOCHS     = 15
LR         = 1e-4    # small learning rate for fine-tuning
VAL_SPLIT  = 0.15    # 15% of training images for validation
SEED       = 42      # fixed seed = repeatable results
MEAN, STD = [0.485, 0.456, 0.406], [0.229, 0.224, 0.225]   # ImageNet statistics
```

All settings sit in one place so they are easy to change and explain. The code uses a GPU automatically if one exists, otherwise the CPU.

Step 2: load the data

```
if not os.path.isdir(DATA_DIR):
    with zipfile.ZipFile(DATA_ZIP) as z:
        z.extractall(".")

TRAIN_DIR = os.path.join(DATA_DIR, "train")
TEST_DIR  = os.path.join(DATA_DIR, "test")
```

This unzips the dataset once. The next cells count images per class and plot a bar chart and sample images (saved as `class_distribution.png` and `samples.png`).

Step 3: preprocessing, augmentation and the train/validation split

```
train_tf = transforms.Compose([
    transforms.Resize((IMG_SIZE + 32, IMG_SIZE + 32)),
    transforms.RandomResizedCrop(IMG_SIZE, scale=(0.8, 1.0)),
    transforms.RandomRotation(10),
    transforms.ColorJitter(brightness=0.2, contrast=0.2),
    transforms.ToTensor(),
    transforms.Normalize(MEAN, STD),
])
eval_tf = transforms.Compose([
    transforms.Resize((IMG_SIZE, IMG_SIZE)),
    transforms.ToTensor(),
    transforms.Normalize(MEAN, STD),
])

full_train_aug  = datasets.ImageFolder(TRAIN_DIR, transform=train_tf)
full_train_eval = datasets.ImageFolder(TRAIN_DIR, transform=eval_tf)
test_ds         = datasets.ImageFolder(TEST_DIR,  transform=eval_tf)

train_idx, val_idx = train_test_split(idx, test_size=VAL_SPLIT,
                                      stratify=full_train_aug.targets, random_state=SEED)
```

- `ImageFolder` reads the class name from each folder name, so no separate label file is needed. It also converts every image to RGB.
- `train_tf` applies augmentation; `eval_tf` only resizes and normalises.
- `stratify=` keeps the same class proportions in the training and validation parts.
- `DataLoader` then feeds the images in batches of 16, shuffling the training data each epoch.

Step 4: build the model

```
def build_model(num_classes, pretrained=True):
    weights = models.ResNet50_Weights.IMAGENET1K_V2 if pretrained else None
    net = models.resnet50(weights=weights)
    for name, p in net.named_parameters():
        p.requires_grad = name.startswith("layer4")      # freeze all but layer4
    net.fc = nn.Sequential(nn.Dropout(0.3),
                           nn.Linear(net.fc.in_features, num_classes))  # new 3-class
head
    return net
```

`requires_grad = False` freezes a layer so its weights do not change during training. Only `layer4` and the new `fc` head learn.

Step 5: loss, optimiser and training loop

```
counts = Counter(np.array(full_train_aug.targets)[train_idx])
class_weights = torch.tensor([len(train_idx) / (len(CLASSES) * counts[i])
                              for i in range(len(CLASSES))])

criterion = nn.CrossEntropyLoss(weight=class_weights)
optimizer = torch.optim.AdamW([p for p in model.parameters() if p.requires_grad],
                              lr=LR, weight_decay=1e-4)
scheduler = torch.optim.lr_scheduler.CosineAnnealingLR(optimizer, T_max=EPOCHS)

for epoch in range(1, EPOCHS + 1):
    tl, ta = run_epoch(train_dl, train=True)    # learn
    vl, va = run_epoch(val_dl, train=False)     # check on validation
    scheduler.step()
    if va >= best_acc:
        best_acc, best_state = va, copy.deepcopy(model.state_dict())

model.load_state_dict(best_state)   # keep the best epoch
```

- **Class weight formula:** total images ÷ (number of classes × images in that class). Rarer classes get a higher weight.
- `run_epoch` does the forward pass, computes the loss and, in training mode, runs `loss.backward()` and `optimizer.step()` to update the weights.
- After every epoch the model is checked on validation data, and the best version is saved. The loss and accuracy curves are saved as `training_curves.png`.

Step 6: evaluate on the test set

```
model.eval()
with torch.no_grad():
    for x, y in test_dl:
        y_pred += model(x.to(DEVICE)).argmax(1).cpu().tolist()
        y_true += y.tolist()

test_acc = accuracy_score(y_true, y_pred)
print(classification_report(y_true, y_pred, target_names=CLASSES, digits=3,
zero_division=0))
cm = confusion_matrix(y_true, y_pred)
```

`model.eval()` switches off dropout; `torch.no_grad()` skips gradient tracking, making prediction faster. `argmax` picks the class with the highest score.

Step 7: save and register the model in Azure ML

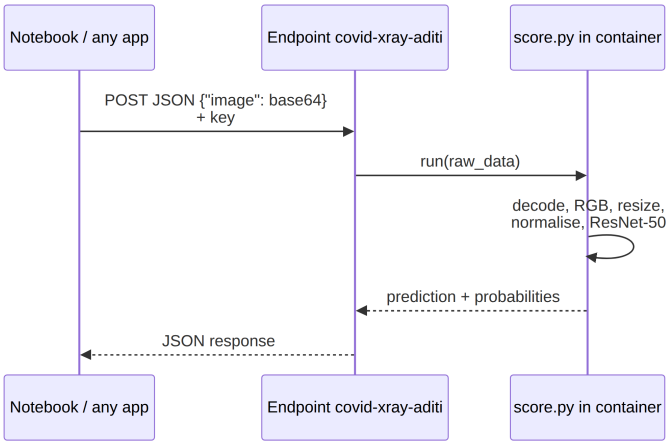
```
torch.save(model.state_dict(), os.path.join(MODEL_DIR, "covid_resnet50.pt"))
json.dump({"classes": CLASSES, "img_size": IMG_SIZE, "mean": MEAN, "std": STD,
           "arch": "resnet50", "test_accuracy": round(test_acc, 4)}, f, indent=2)

ml_client = MLClient.from_config(credential=DefaultAzureCredential())
registered = ml_client.models.create_or_update(Model(
    path=MODEL_DIR, name="covid-xray-resnet50", type=AssetTypes.CUSTOM_MODEL,
    tags={"framework": "pytorch", "test_accuracy": f"{test_acc:.4f}"},
))
```

- The weights go into `covid_resnet50.pt`. The class names and preprocessing settings go into `labels.json`, so the endpoint preprocesses new images exactly as in training.
- `MLClient.from_config` connects to the workspace; `DefaultAzureCredential` uses the logged-in account.
- `create_or_update` uploads the model folder to the registry. Each run creates a new version (1, 2, 3...).

6. Code walkthrough: deployment (score.py, conda.yml, 02_deploy_endpoint.ipynb)

6.1 How a prediction request works



The image travels as base64 text inside JSON, because a REST API sends text, not raw files. The key proves the caller is allowed to use the endpoint.

6.2 The scoring script (src/score.py)

Azure ML calls `init()` once when the container starts, and `run()` for every request.

```
def build_model(num_classes):
    net = models.resnet50(weights=None)
    net.fc = nn.Sequential(nn.Dropout(0.3), nn.Linear(net.fc.in_features, num_classes))
    return net

def init():
    global model, classes, preprocess
    model_root = os.getenv("AZUREML_MODEL_DIR", ".")
    meta_path = glob.glob(os.path.join(model_root, "**", "labels.json"), recursive=True)
[0]
    weights_path = glob.glob(os.path.join(model_root, "**", "*.pt"), recursive=True)[0]
    meta = json.load(open(meta_path))
    classes = meta["classes"]
    model = build_model(len(classes))
    model.load_state_dict(torch.load(weights_path, map_location="cpu"))
    model.eval()
    preprocess = transforms.Compose([
        transforms.Resize((meta["img_size"], meta["img_size"])),
        transforms.ToTensor(),
        transforms.Normalize(meta["mean"], meta["std"]),
    ])

def run(raw_data):
    data = json.loads(raw_data)
    images = data.get("images") or [data["image"]]           # one image or a batch
    batch = torch.stack([preprocess(_decode(s)) for s in images])
    with torch.no_grad():
        probs = torch.softmax(model(batch), dim=1)
    return [{"prediction": classes[int(p.argmax())],
             "confidence": round(float(p.max()), 4),
             "probabilities": {c: round(float(v), 4) for c, v in zip(classes, p)}}
            for p in probs]
```

- `build_model` must match the training architecture exactly, or the saved weights will not load.
- `AZUREML_MODEL_DIR` is where Azure copies the registered model inside the container.
- `_decode` turns the base64 text back into an image and converts it to RGB, so **any new JPG or PNG X-ray** is handled, including grayscale ones.
- Preprocessing is read from `labels.json`, so new images are treated exactly like training images.
- `softmax` turns the raw scores into probabilities that add up to 1. The highest one is the prediction and its value is the confidence.
- Wrong input returns a clear error message instead of crashing.

6.3 The environment (src/conda.yml)

```
dependencies:
  - python=3.10
  - pip:
      - --extra-index-url https://download.pytorch.org/whl/cpu
      - torch==2.3.1+cpu
      - torchvision==0.18.1+cpu
      - pillow
      - numpy<2
      - azureml-inference-server-http
```

The CPU-only PyTorch build keeps the container small, because the endpoint runs on a CPU machine. `azureml-inference-server-http` is the web server that receives requests and calls `run()`.

6.4 Deploying the endpoint

```
ENDPOINT_NAME = "covid-xray-aditi"          # fixed name, so re-running reuses it
model = ml_client.models.get("covid-xray-resnet50", label="latest")

ml_client.online_endpoints.begin_create_or_update(
    ManagedOnlineEndpoint(name=ENDPOINT_NAME, auth_mode="key")).result()

env = Environment(name="covid-xray-inference-env", conda_file="src/conda.yml",
                  image="mcr.microsoft.com/azureml/openmpi4.1.0-ubuntu22.04:latest")
deployment = ManagedOnlineDeployment(
    name="blue", endpoint_name=ENDPOINT_NAME, model=model, environment=env,
    code_configuration=CodeConfiguration(code="src", scoring_script="score.py"),
    instance_type="Standard_DS1_v2", instance_count=1)
ml_client.online_deployments.begin_create_or_update(deployment).result()

ep = ml_client.online_endpoints.get(ENDPOINT_NAME)
ep.traffic = {"blue": 100}                    # send all requests to this deployment
ml_client.online_endpoints.begin_create_or_update(ep).result()
```

- An **endpoint** is the stable URL. A **deployment** (`blue`) is the model version running behind it. Traffic decides which deployment answers.
- `label="latest"` always deploys the newest trained model version.
- Azure builds a Docker image from the base image plus `conda.yml`, which takes 10–20 minutes the first time.
- `Standard_DS1_v2` (1 vCPU) was chosen to fit the student subscription's 6-vCPU quota (see Challenges).

6.5 Predicting a new image

```
def predict_new_image(image_path, endpoint_name=ENDPOINT_NAME):
    with open(image_path, "rb") as f:
        payload = {"image": base64.b64encode(f.read()).decode("utf-8")}
    with open("new_request.json", "w") as f:
        json.dump(payload, f)
    result = json.loads(ml_client.online_endpoints.invoke(
        endpoint_name=endpoint_name, request_file="new_request.json"))[0]
    print(f"Prediction: {result['prediction']}  (confidence {result['confidence']:.0%})")
    if result["confidence"] < 0.6:
        print("   Low confidence - the model is unsure about this image.")
    return result

predict_new_image("new_xray.jpeg")
```

To test any new X-ray: upload it to the `covid-xray` folder, set its file name and run the cell. The same endpoint can also be called from any app or Postman with a plain HTTP POST to its scoring URL, using the key from the endpoint's **Consume** tab.

7. Results

The model classified **62 of 66 unseen test images correctly (93.94% accuracy)**. The live endpoint got **6 of 6** sample images right and correctly classified a new image.

7.1 Test-set performance

Metric	Value
Test accuracy	93.94% (62 / 66)
Covid images correctly identified	25 of 26 (recall 96.2%)
Covid images misclassified	1, predicted as Normal; none as Viral Pneumonia
Reproducibility	Retraining with seed 42 gave the same 93.94%

Still to add from my notebook: the Normal and Viral Pneumonia rows of the confusion matrix (`outputs/confusion_matrix.png`), and per-class precision, recall and F1 (`outputs/classification_report.txt`).

Reading the result: Covid, the most important class, was almost always caught, with only 1 of 26 cases missed. The other 3 errors are among the 40 Normal and Viral Pneumonia images; the confusion matrix shows which way they went.

7.2 Endpoint test (live REST API)

Actual class	Predicted class	Confidence
Covid	Covid	0.97
Covid	Covid	0.99
Normal	Normal	0.86
Normal	Normal	0.99
Viral Pneumonia	Viral Pneumonia	0.70
Viral Pneumonia	Viral Pneumonia	0.97

All 6 were correct. The lower confidence of 0.70 on one Viral Pneumonia image shows the model still expresses uncertainty on harder cases.

7.3 New image prediction

A test X-ray not used in the endpoint test was saved as `new_xray.jpeg` and sent to the endpoint:

Class	Probability
Viral Pneumonia (predicted, correct)	96.5%
Normal	2.8%
Covid	0.7%

This shows the deployed service works end to end for a new image: upload, encode, send, preprocess, predict and return.

7.4 Charts saved for the report (outputs/ folder)

- `class_distribution.png`: images per class in train and test.
- `samples.png`: 4 example X-rays per class.
- `training_curves.png`: training vs validation loss and accuracy per epoch.
- `confusion_matrix.png`: test-set confusion matrix.
- `classification_report.txt`: precision, recall and F1 per class.

8. Challenges faced and how I solved them

The hardest part was not the model but working within cloud limits: CPU quota, package setup and endpoint management.

#	Problem	Cause	Solution
1	`ModuleNotFoundError: No module named 'torch'`	The notebook kernel (Python 3 ipykernel) did not include PyTorch	Installed CPU PyTorch into the kernel's own Python with `! {sys.executable} -m pip install torch torchvision`, then restarted the kernel
2	Model registration needed a login	The notebook had not signed in to Azure	Clicked **Authenticate** on the compute instance so the Azure SDK could use my account
3	Deployment failed: "Not enough quota for Standard_DS3_v2"	The student subscription allows 6 vCPUs; the compute instance used 4, and a DS3_v2 deployment needs 8 (Azure reserves double for upgrades)	Switched the endpoint to **Standard_DS1_v2** (1 vCPU, needs 2), which fits the remaining quota
4	Several extra endpoints (`covid-xray-5f4c2b`, `a19974`, `8b343c`, `4e0fea`)	The notebook added a random suffix to the name every time the cell ran	Changed the code to one **fixed name** (`covid-xray-aditi`) that is reused if it already exists
5	Working endpoint got deleted; traffic update failed with "endpoint is in deleting state"	**Run all** also ran the last clean-up cell, which deletes the endpoint	Disabled the clean-up cell (commented out), so Run all is now safe; redeployed
6	Traffic showed `blue (0%)`	A deployment gets no requests until traffic is assigned to it	Set traffic to `{"blue": 100}` in code (and via **Update traffic** in Studio)
7	Scoring files not found	`score.py` and `conda.yml` were uploaded outside the `src` folder	Moved them into `src/`, where the deployment code looks for them

What I learned

- Cloud resources have quotas, so the VM size must be chosen to fit the subscription.
- Deployment code should be **idempotent**, meaning running it twice gives the same result, with no duplicates.
- Destructive actions such as delete should never sit in a notebook that is run with Run all.
- Endpoints cost money every hour they exist, so delete them and stop compute after use.

9. Limitations, future scope and conclusion

Limitations

- **Small dataset:** 251 training and 66 test images. The accuracy is promising, but a larger and more varied dataset is needed before real-world use.
- **Always picks one of 3 classes:** the model has no "not an X-ray" or "other disease" option, so it will label any image as one of the three.
- **No explanation:** it does not show which part of the lung led to its decision.
- **Not clinically validated:** it is a student project, not a medical device, and must not replace a doctor.
- **Small CPU endpoint:** `Standard_DS1_v2` is enough for a demo, but not for many users at once.

Future scope

1. Train on a larger public dataset (thousands of images) and add more classes, such as bacterial pneumonia and tuberculosis.
2. Add **Grad-CAM heatmaps** to highlight the lung regions behind each prediction.
3. Compare with other architectures (DenseNet-121, EfficientNet) and with **Azure AutoML for Images**, which needs no code.
4. Build a simple web page or Streamlit app where a user uploads an X-ray and sees the result.
5. Add an "uncertain, please review" rule for low-confidence predictions (below 60%) so a doctor always checks those.
6. Use a GPU and larger endpoint instances if the service had real users.

Conclusion

This project covers the full machine learning lifecycle on Azure: data preparation, transfer-learning training, evaluation, model registration, deployment as a REST endpoint and prediction on new images. The ResNet-50 model reached 93.94% test accuracy, caught 25 of 26 Covid cases, and classified new X-rays correctly through the live endpoint. Solving the quota and deployment problems also showed how cloud cost and resource limits shape real ML engineering decisions.

10. Viva preparation: likely questions and answers

Q1. What problem does your project solve? It automatically classifies a chest X-ray as Covid, Normal or Viral Pneumonia, and exposes the model as a cloud API so any app can get a prediction.

Q2. Why did you use Azure Machine Learning? It gives cloud compute for training, a model registry for versioning, and managed endpoints for deployment, all in one workspace. I didn't need a powerful laptop, and the model can be used from anywhere.

Q3. What is transfer learning and why did you use it? Reusing a model trained on a large dataset (ImageNet) and fine-tuning it on my task. With only 251 images, training from scratch would overfit. Pretrained features gave 93.94% accuracy.

Q4. Why ResNet-50? It is a proven, widely used CNN. Its skip connections make a 50-layer network train reliably, and pretrained weights are readily available in torchvision.

Q5. Why freeze most layers? Early layers detect general features like edges and textures, which are useful for X-rays too. Freezing them cuts the number of weights to learn and reduces overfitting. Only `layer4` and the new head are trained.

Q6. What is data augmentation? Why no horizontal flip? Randomly changing training images (crop, rotate, brightness) so the model sees more variety. I avoided flipping because a mirrored chest X-ray (heart on the wrong side) is not realistic.

Q7. How did you handle class imbalance? Covid has 111 training images versus 70 for each other class. I used class weights in the cross-entropy loss, so mistakes on the smaller classes cost more.

Q8. Why split off a validation set when you already have a test set? Validation picks the best epoch during training. The test set must stay unseen until the end, otherwise the accuracy would be over-optimistic.

Q9. Which metric matters most here? Recall for Covid, because missing a real Covid case is more dangerous than a false alarm. My model's Covid recall is 25/26 = 96.2%.

Q10. What are softmax and confidence? Softmax turns the model's 3 raw scores into probabilities that add up to 1. The highest probability is the prediction, and its value is the confidence.

Q11. What is the difference between an endpoint and a deployment? The endpoint is the stable URL and authentication. A deployment is a specific model version running behind it on a VM. Traffic rules decide which deployment answers, which allows safe updates (blue/green).

Q12. What does score.py do? `init()` loads the model once when the container starts. `run()` receives each request, decodes the base64 image, preprocesses it the same way as training, predicts and returns JSON.

Q13. How does the model handle a new image? The image is converted to base64 and sent as JSON to the endpoint. `score.py` converts it to RGB, resizes to 224×224, normalises, runs ResNet-50 and returns the class and probabilities. I tested it with a new X-ray: Viral Pneumonia, 96.5%.

Q14. Why Standard_DS1_v2 for the endpoint? The student subscription has a 6-vCPU quota. The compute instance used 4, and Azure reserves double capacity for a deployment, so only a 1-vCPU VM fit.

Q15. What would you improve? A larger dataset, Grad-CAM heatmaps for explainability, comparing with AutoML or other CNNs, a simple web app front end, and a low-confidence review rule.

Q16. Can this be used in hospitals? Not as it is. It was trained on only 317 images and never clinically validated. It is a proof of concept showing how such a tool can be built and deployed.